

指针变量的**赋值**和**初始化**是两个重要的概念，但是除了指向已经存在的变量（如 a），指针还可以指向**系统动态申请的内存空间**。这是 C 语言指针最强大的地方，也是链表、树等数据结构的基础。这通常配合 malloc 函数使用。

```
1 #include <stdio.h>
2 #include <stdlib.h> // 必须包含这个头文件才能使用 malloc、free
3
4 int main() {
5     // 【动态内存分配】
6     // malloc 向操作系统申请 4 个字节 (sizeof(int)) 的空间
7     // 并把这块空间的首地址返回给指针 p
8     int* p = (int*)malloc(sizeof(int));
9
10    // 检查是否申请成功
11    if (p != NULL) {
12        // 此时 p 指向的是一块“无名”的内存，而不是某个变量 a
13        *p = 300; // 向这块内存里写入数据
14
15        printf("动态分配方式:\n");
16        printf("内存中的值: %d\n", *p);
17
18        // 用完了一定要手动释放内存，否则会内存泄漏
19        free(p);
20        p = NULL; // 释放后置空，防止野指针
21    }
22
23    return 0;
24 }
```

再补充一种：C99 复合字面量 (扩展知识)

如果你的编译器支持 C99 标准 (现代编译器大多支持)，还可以直接让指针指向一个“匿名的变量或数组”。这算是另一种“初始化”。

```
1 #include <stdio.h>
2
3 int main() {
4     // 【指向匿名变量】
5     // (int){1000} 在内存中创建了一个临时的整型数据
6     // p 直接指向它
7     int* p = &(int) { 1000 };
8
9     printf("匿名变量方式: %d\n", *p);
10
11    return 0;
12 }
```

现在我将对刚刚的**系统动态申请的内存空间**代码，分三个部分深入剖析：**代码深度解析、动态内存的作用与注意点、408 考研中的核心考点与应用**。

第一部分：代码深度解析 (逐行拆解)

1. 核心语句：malloc 申请内存 (malloc 申请的地址是随机的，但空间是连续的)

```
1 int *p = (int*)malloc(sizeof(int));
```

这句话发生了四件事，非常重要：

1. sizeof(int): 计算出 int 类型占用的字节数 (在大多数现代计算机上是 4 字节)。这样做的好处是**可移植性**，即使把代码移植到 int 占 8 字节的机器上，代码也不用改。
2. malloc(sizeof(int)): 向操作系统 (实际上是 C 运行时库的内存管理器) 发出请求：“请给我 4 个字节连续的内存空间”。
 - 如果申请成功，malloc 返回这块内存首字节的地址 (**通用指针 void* 类型**)。
 - 如果申请失败 (比如内存耗尽)，**返回 NULL (空指针)**。
3. (int*) **强制类型转换**:
 - malloc 返回的是 void* (万能指针)，它只告诉你地址，没告诉你这块地址怎么用 (是存整数还是存字符?)。
 - (int*) 告诉编译器：“把这个地址看作是**指向整数的指针**”。这样，指针运算 (如 p+1) 才能正确移动 4 个字节。
 - 注：在 C 语言标准中，void* 可以自动转换为其他指针，但这步强制转换是 C++ 的习惯，写上它可以增加代码兼容性。
4. **赋值给 p**: 指针变量 p 此时存储了这块堆内存的地址。

2. 空指针检查

```
1 if (p != NULL)
```

知识点：这是防御性编程。**绝对不要假设内存申请一定成功**。如果内存不足，malloc 返回 NULL，此时如果直接执行 *p = 300，会导致“段错误”或程序崩溃。

3. 使用内存

```
1 *p = 300;
```

知识点：这叫**解引用**。程序通过指针 p 找到堆上的那块无名内存，并把数值 300 写进去。此时，这块内存才真正有了意义。

4. 释放内存

```
1 free(p);
```

知识点：这是动态内存管理的底线。“**借了要还**”。

原理：free(p) 并不改变指针变量 p 本身的值 (p 里依然存着那个地址)，它只是告诉操作系统：“这块地盘我不用了，你可以回收去给别人用”。

5. 置空指针

```
1 p = NULL;
```

知识点：这步叫“切断联系”。防止后续代码不小心再使用 p 去访问已经释放的内存（野指针访问）。虽然 free 后程序紧接着就结束了，但在大型项目中，这步至关重要。

第二部分：动态内存分配的作用与注意点

1. 为什么要用动态内存？（作用）

你可能会问：“我直接定义变量 int a = 300; 不香吗？为什么要这么麻烦？”

原因一：克服栈空间的限制。

普通变量 (int a) 定义在栈上。栈空间很小（通常只有 1MB - 8MB）。如果你想开一个巨大的数组（比如存 100 万个学生信息），直接定义 int arr[1000000] 会瞬间撑爆栈，导致程序崩溃。而堆空间非常大，受限于电脑的物理内存。

原因二：灵活性（运行时决定）。

普通数组的大小必须在写代码时就确定（静态）。

动态内存允许在程序运行时根据用户输入来决定大小。例如：用户说“我要存 5 个数”，你就 malloc(5 * sizeof(int)); 用户说“我要存 5000 个”，你就 malloc(5000 * sizeof(int))。这是 C 语言实现变长数组的经典方式。

2. 致命注意点（坑）

内存泄漏：

如果你 malloc 了内存，却忘了 free，这块内存就会一直被占用，直到程序结束。如果这是个服务器程序，运行了几天几夜，内存会被慢慢吃光，最终导致系统崩溃。

野指针：

指针指向的内存已经被 free 了，但指针还在指向那里。此时再去读写，就像去住一间已经退房的酒店房间，后果不可预测。

悬空指针：

多个指针指向同一块内存，free 了其中一个，另一个指针就变成了悬空指针。

第三部分：408 考研中的运用与考点（重点突破）

在计算机考研 408 数据结构部分，动态内存分配是**数据结构实现的基石**。它不会单独考 malloc 的语法，而是融合在算法题和概念题中。

1. 数据结构实现的核心机制

在 408 的数据结构大题（算法题）中，当你需要实现链表、树、图时，必须使用 malloc。

链表：

当你执行 Insert 操作（插入节点）时，本质上就是在调用 malloc 申请一个 LNode 大小的空间。代码范式：

```
1 LNode *s = (LNode*)malloc(sizeof(LNode)); // 申请节点
2 s->data = x;
3 s->next = p->next;
4 p->next = s;
```

二叉树：

创建新节点同样需要 malloc。

图：

邻接表存储方式中，顶点的边链表节点也是动态生成的。

2. 栈与堆的区别（选择题高频考点）

408 常考内存分区，务必区分清楚：

栈：

存放局部变量、函数参数。

特点：由系统自动管理（函数结束自动释放），空间小，速度快。

考点：递归调用的深度受限于栈的大小。

堆：

存放 malloc 申请的动态内存。

特点：由程序员手动管理（手动申请和释放），空间大，速度相对较慢（需要查找空闲链表）。

考点：容易产生内存泄漏。

3. 常见考题陷阱

陷阱 A：指针与指向内容的混淆

题目问：free(p) 释放的是什么？

正确答案：释放的是指针 p 指向的内存空间，而不是释放指针变量 p 本身。变量 p 本身依然存在于栈上，直到它所在的函数结束。

陷阱 B：函数内申请内存返回

错误写法：

```
1 int* createInt() {
2     int a = 10;
3     return &a; // 错！返回了栈上变量的地址
4 }
```

正确写法：

```
1      int* createInt() {
2          int *p = (int*)malloc(sizeof(int));
3          *p = 10;
4          return p; // 对! 返回堆上地址, 堆在函数结束后依然存在
5      }
```

考点: 理解函数调用结束后, 栈帧销毁, 局部变量 a 消失; 但堆内存不会自动消失, 必须手动 free。

4. 408 算法题建议

在 408 算法题 (通常 10-15 分) 中:

- 1.如果你写了链表或树的插入操作, 必须体现 malloc 的过程。
- 2.虽然在考试代码中, 阅卷老师通常不强制要求写 free (因为只是片段代码), 但在思路分析或注释中提到 “注意释放内存防止泄漏” 会是一个加分项, 体现你的工程素养。
- 3.书写规范: 考试时建议使用 `p = (LNode *)malloc(sizeof(LNode));` 这种标准写法, 并紧跟一个 `if(p==NULL) exit(0);` 的判空逻辑 (虽然通常题目不要求写, 但这能体现严谨性)。